

LAB | ETL, ELT, and Pipeline Architecture

Week 3 — Day 1: ETL, ELT, and Pipeline Architecture

Q1.

Source -> files received with data.

Ingestion -> process of taking the data file from the source into the pipeline

Raw -> stores data “as is”, before cleaning takes place

Staging -> basic cleaning and validation such as data formatting

Transformation -> process of applying business rules and combining data sets

Analytics -> stores data ready for reports and dashboards

Q. 2 — Compare ETL and ELT

Complete the comparison.

Area	ETL	ELT
When transformation happens	Before loading	After loading
Where transformation usually runs	In a separate processing tool or script	Inside the target database or warehouse
Suitable example	Clean a CSV with Python, then load it	Raw data is loaded, then SQL transforms it

Q. 3 — Practical: Rewrite an ETL Flow as ELT:

Extract JSON files



Load raw data into the warehouse



Clean and standardize the data inside the warehouse using SQL



Prepare data for reports and dashboards

Q.4 - Given the target of an operational database that contains only valid records, I would recommend ETL, so that any invalid rows or incorrect format or null values are not reflected in the db.

Q. 5 — MCQ: Pipeline Purpose: B

Q. 6 — MCQ: Raw Layer: B

Q. 7 — MCQ: ETL Order: A

Q. 8 — MCQ: ELT Transformation: B

Q. 9 — MCQ: Best Fit: A

Q. 10 — Batch and Streaming Processing

Use case	Choice
Daily sales report	batch
Payment-fraud alert	streaming
Hourly stock snapshot	batch
Immediate delivery notification	streaming

Q. 11 — Practical: Simulate Batch and Streaming Events

import time

from datetime import datetime

```
orders = [  
    {"order_id": 1001, "amount_gbp": 24.99},  
    {"order_id": 1002, "amount_gbp": 18.50},  
    {"order_id": 1003, "amount_gbp": 42.75},  
    {"order_id": 1004, "amount_gbp": 12.00},  
    {"order_id": 1005, "amount_gbp": 31.20},  
]
```

```
print("BATCH")  
for order in orders:  
    print(order)
```

```
print("\nSTREAMING SIMULATION")
for order in orders:
    print(datetime.now().isoformat(timespec="seconds"), order)
    time.sleep(1)
```

Q. 12 — Practical: Ingest a Customer File into Raw Storage

```
customer_id,name,email,city
1, Amelia Clarke ,AMELIA@EXAMPLE.COM,Manchester
2,Oliver Bennett,oliver@example.com,Bristol
```

```
from pathlib import Path
import csv
import shutil

source = Path("data/source/customers.csv")
raw = Path("data/raw/customers.csv")
raw.parent.mkdir(parents=True, exist_ok=True)

shutil.copyfile(source, raw)

with raw.open("r", newline="", encoding="utf-8") as file:
    row_count = sum(1 for _ in csv.DictReader(file))

print(f"Raw ingestion complete: {row_count} rows")
```

Q. 13 — Practical: Clean Raw Customers into Staging

```
from pathlib import Path
import csv

source = Path("data/raw/customers.csv")
target = Path("data/staging/customers.csv")
target.parent.mkdir(parents=True, exist_ok=True)

with source.open("r", newline="", encoding="utf-8") as source_file:
    rows = list(csv.DictReader(source_file))

for row in rows:
    row["name"] = row["name"].strip()
    row["email"] = row["email"].strip().lower()

fieldnames = ["customer_id", "name", "email", "city"]
```

```
with target.open("w", newline="", encoding="utf-8") as target_file:
    writer = csv.DictWriter(target_file, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rows)
```

```
for row in rows:
    print(row)
```

Q. 14 — MCQ: Daily Report: A

Q. 15 — MCQ: Immediate Alert: B

Q. 16 — MCQ: Ingestion: A

Q. 17 — MCQ: Staging A

Q. 18 — MCQ: Layer Benefit: A

Q. 19 — Pipeline Dependencies

```
ingest_customers
    ↓
clean_customers
    ↓
build_customer_summary
    ↓
publish_report
```

If clean_customers step fails, the following instructions should stop because they depend on cleaned data. Running all tasks independently could produce an inconsistent and incorrect summary or report due to missing or invalid input.

Q. 20 — Practical: Stop a Pipeline after Failure

```
from pathlib import Path
```

```
def ingest():
    print("ingest: success")
```

```

def clean():
    raw_file = Path("data/raw/customers.csv")
    if not raw_file.exists():
        raise FileNotFoundError(raw_file)
    print("clean: success")

def publish():
    print("publish: success")

steps = [
    ("ingest", ingest),
    ("clean", clean),
    ("publish", publish),
]

for step_name, step_function in steps:
    try:
        step_function()
    except Exception as error:
        print(f"Pipeline stopped at {step_name}: {error}")
        break
else:
    print("Pipeline finished successfully")

```

Q. 21 — Practical: Make Ingestion Safe to Re-run

```

from pathlib import Path
import hashlib
import shutil

```

```

source = Path("data/source/customers.csv")
target = Path("data/raw/customers.csv")
target.parent.mkdir(parents=True, exist_ok=True)

```

```

def file_hash(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as file:
        for chunk in iter(lambda: file.read(8192), b""):
            digest.update(chunk)
    return digest.hexdigest()

```

```

if target.exists() and file_hash(source) == file_hash(target):
    print("Ingestion skipped: raw file is already current")
else:
    shutil.copyfile(source, target)
    print("Ingestion complete: raw file copied")

```